

1. Memory Efficient Doubly Linked List – Introduction

In conventional implementation, we need to keep a forward pointer to the next item on the list and a backward pointer to the previous item. That means elements in doubly linked list implementations consist of data, a pointer to the next node and a pointer to the previous node in the list as shown below:

```
typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
} Node;
```

But in XOR (Memory Efficient) List we have :

```
typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;

Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}
```

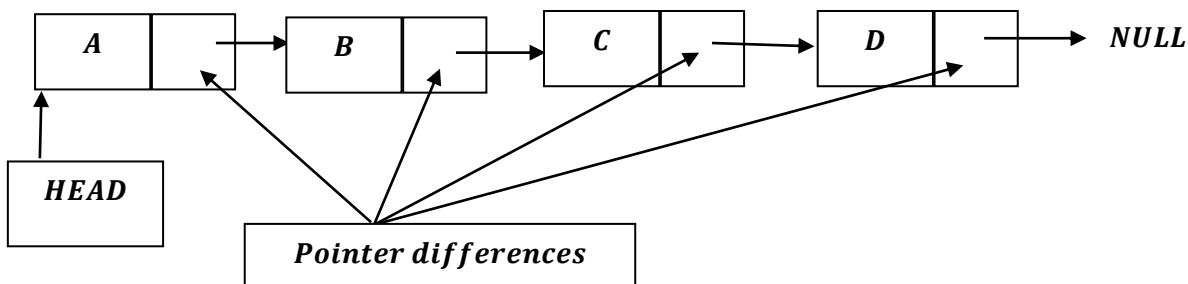
The *'ptrdiff'* pointer field contains the difference between the pointer to the next node and the pointer to the previous node. The pointer difference is calculated by using exclusive – or ($\oplus$) operation.

$$\text{ptrdiff} = \text{pointer to previous node} \oplus \text{pointer to next node}.$$

The *'ptrdiff'* = *'npx'* as NPX is stands for NEXT – PREVIOUS XOR.

The same *'ptrdiff'* definition for *'NPX'* : the difference between the pointer to the next node and the pointer to the previous node.

The *'ptrdiff'* of the start node(head node) is the $\oplus$ of NULL and next node (next node to head). Similarly , the *'ptrdiff'* of end node is the $\oplus$ of previous node(previous to end node) and NULL.



In the example above,

- The next pointer of A is : $\text{NULL} \oplus B$
- The next pointer of B is: $A \oplus C$
- The next pointer of C is : $B \oplus D$
- The next pointer of D is $C \oplus \text{NULL}$

How does it work?

To find the answer to this question let us consider the properties of $\oplus$.

- $X \oplus X = 0$
- $X \oplus 0 = X$
- $X \oplus Y = Y \oplus X$ (Symmetric)
- $(X \oplus Y) \oplus Z = X \oplus (Y \oplus Z)$ (transitive)

For the example above, let us assume that we are at C node and want to move to B. We know that C's ptrdiff is defined as $B \oplus D$. If we want to move to B, performing $\oplus$ on C's ptrdiff with D would give B. This due to the fact that:

$$(B \oplus D) \oplus D = B \text{ (since , } D \oplus D = 0 \text{)}$$

Similarly , if we want to move to D, then we have to apply $\oplus$ to C's ptrdiff with D would give B. This is due to fact that:

$$(B \oplus D) \oplus B = D \text{ (since, } B \oplus B = 0 \text{)}$$

Form the above discussion, we can see that just by using a single pointer, we can move back and forth.

A memory – efficient implementation of a doubly linked list is possible with minimal compromising of timing efficiency.
